1. Problem statement

Build a REST API for a simplified multi-tenant messaging platform. External providers send webhook events such as message-created and delivery-status updates. Providers may retry events, events may arrive out of order, and the service must avoid duplicate business effects.

No frontend or real WhatsApp/Meta integration is required. Simulate webhook calls locally.

2. Required functionality

A. Create and retrieve messages

POST /api/v1/messages
Accept JSON containing tenantId, conversationId, to, and text. Validate required fields and reasonable length limits. Generate a message ID, persist the message with status QUEUED, and return a consistent response.

GET /api/v1/conversations/:conversationId/messages?limit=20&cursor=...

Return messages in stable chronological order with pagination. Enforce tenant isolation. For this exercise, x-tenant-id may represent the authenticated tenant; explain the assumption.

B. Idempotent provider webhook

POST /api/v1/webhooks/provider

Accept eventId, messageId, status (SENT, DELIVERED, READ, FAILED), and occurredAt.

- Repeated eventId must not create duplicate event records or apply the same effect twice.
- Validate payloads and return clear 4xx responses for invalid data.
- Prevent old events from moving a message backwards (for example READ to SENT). Define and document status-transition rules and out-of-order handling.
- Persist the event or work item before acknowledging success. A real broker is optional; document limitations if using in-process asynchronous work.
- Handle unknown message IDs intentionally, such as a clear 404 or a documented unmatched-event strategy.

C. Read status and health

GET /api/v1/messages/:messageId
returns message details, current status and relevant timestamps.

GET /health
returns service health; optionally include a database connectivity check without exposing secrets.

D. Persistence and setup

Use PostgreSQL, SQLite, or another persistent database. Include migrations or reproducible schema setup. In-memory-only storage does not satisfy the persistence requirement.

3. Example payloads

Create message

{ "tenantId":"tenant-a", "conversationId":"conv-100", "to":"+919876543210", "text":"Your Order has shipped" }

Webhook event

{ "eventId":"evt-001", "messageId":"msg-123", "status":"DELIVERED", "occurredAt":"2026-10-16T08:30:00.000Z" }

Exact response shapes are your choice. Document them in the README and keep them consistent.

4. Acceptance criteria

- App starts from documented commands; database setup is reproducible.
- Message persists after server restart and can be retrieved.
- Invalid payloads return appropriate 4xx responses, not generic 500s.
- Duplicate eventId is safely handled.
- Status progression is enforced, including an out-of-order event case.
- Conversation history has stable pagination and tenant isolation.
- Automated tests cover core success paths and at least two edge/failure cases.
- Logs help debugging without unnecessarily exposing secrets or message content.
- README documents schema, API examples, assumptions, trade-offs, and limitations.

5. Optional stretch goals

- Add a worker queue with retry/backoff and dead-letter strategy.
- Protect simultaneous status updates with transactions or optimistic concurrency.
- Add structured logs, correlation IDs, and metrics for volume/failures.
- Provide Docker Compose setup.
- Explain how the design changes at high throughput using a broker and distributed database.

6. Submission requirements

- Source code in a Git repository or ZIP.
- README with exact install, migration, run, and test commands.

- Environment variable example file with placeholder values only.
- Automated tests and sample curl/Postman requests.
- Short design note: idempotency, status transitions, tenant isolation, and production